Ders 10 Genişletilmiş Notu: Seminer Sunumları: Dalgacık Dönüşümü, Sinir Ağları, Model Sıkıştırma, Goldbach Kodları ve D-v2v

Ders 10 Genişletilmiş Notu: Seminer Sunumları: Dalgacık Dönüşümü, Sinir Ağları, Model Sıkıştırma, Goldbach Kodları ve D-v2v

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Yapısı
- 2. Sunum 1: Haar Ayrık Dalgacık Dönüşümü ile Görüntü Sıkıştırma
- 3. Sunum 2: Sinir Ağlarıyla Kayıpsız Veri Sıkıştırma
- 4. Sunum 3: FastText.zip, Metin Sınıflandırma Modellerinin Sıkıştırılması
- 5. Sunum 4: RLE, Aritmetik Kodlama, Punctured Elias ve Goldbach Kodlarının Karşılaştırılması
- 6. Sunum 5: Doğal Dil İşleme için CNN Sıkıştırma: Niceleme ve Budama
- 7. Sunum 6: Dinamik Değişkenden Değişkene Sıkıştırma (D-v2v)
- Örnek Sorular

1. Dersin Yapısı

Bu ders, önceki derslerde işlenen konuların devamı değil, öğrenci seminer sunumlarına ayrılmış bir derstir. Her sunumda bir öğrenci veri sıkıştırma ile ilgili yayımlanmış bir makaleyi özetledi. Hoca, sürenin 15 dakika, en fazla 20 dakika olması gerektiğini söyledi: hem çok uzun dinlemek yorucudur, hem de anlatılan kişinin kendi çalışması değil, bir başkasının çalışmasının özetidir. Her sunumun sonunda hoca makalenin yılını sordu, sorular yöneltti ve yorum yaptı. Toplam altı sunum yapıldı. Konular sırasıyla görüntü sıkıştırmada dalgacık dönüşümü, sinir ağlarıyla kayıpsız sıkıştırma, metin sınıflandırma modellerinin sıkıştırılması, tamsayı kodlarının karşılaştırılması, CNN modellerinin niceleme ve budama ile küçültülmesi ve kelime tabanlı dinamik bir sıkıştırıcıdır.

Bu not her sunumu sunulduğu sırayla, slaytlardaki bilgiler ve sunum sırasındaki açıklamalarla birlikte verir. Sunumların önemli bir kısmı İngilizce yapıldığı ve altyazı bazı yerlerde anlamsız tekrara düştüğü için, anlatım büyük ölçüde ekrandaki slaytlardan kurulmuştur.

2. Sunum 1: Haar Ayrık Dalgacık Dönüşümü ile Görüntü Sıkıştırma

İlk sunumda bir öğrenci, Haar ayrık dalgacık dönüşümü (Haar Discrete Wavelet Transform) ile görüntü sıkıştırmayı anlatan, sunan öğrenciye göre 2020 yılına ait bir makaleyi özetledi.

2.1 Neden dönüşüm?

Bir sinyale dönüşüm uygulanmasının nedeni, ham sinyalde doğrudan okunamayan bilgiyi elde etmektir. En yaygın dönüşüm Fourier dönüşümüdür. Sunan öğrenciye göre frekans yanıtı analizi, zaman sabitleri çok büyük sistemler için önerilmez.

Dalgacık (wavelet) dönüşümü, bir sinyalin hem zaman hem de frekans gösterimini elde etmek için kullanılan matematiksel bir dönüşümdür. Dalgacıkların iki temel özelliği vardır:

- **Ölçek (scale, dilation):** dalgacığın ne kadar gerildiğini ya da sıkıştırıldığını belirler.
- **Konum (location, translation):** dalgacığın zaman ya da uzayda nereye yerleştirildiğini belirler.

Temel fikir, verilen bir ölçek ve konum için sinyalin dalgacıkla ne kadar örtüştüğünü hesaplamaktır.

Slayttaki iki biçim (ekrandan, 00:10:00):

- Sürekli dalgacık dönüşümü (CWT):

$$CWT(a,b) = (f, \psi_{a,b}) = \frac{1}{\sqrt{a}} \int_{-\infty}^{+\infty} f(t)\, \psi^* \left(\frac{t-b}{a} \right) dt$$

- Ayrık dalgacık dönüşümü (DWT):

$$DWT(m,n) = (f, \psi_{m,n}) = a_0^{-m/2} \sum_{k=-\infty}^{+\infty} f(k)\, \psi^*(a_0^{-m} k - n b_0)$$

2.2 Görüntü sıkıştırmada dalgacıklar

Görüntü sıkıştırmada dalgacık kullanmak, negatif olmayan bir eşik değeri t seçmek ve büyüklüğü bu eşiği aşmayan detay katsayılarını sıfır kabul etmek demektir. Daha çok sıfır daha az yer demektir. Ardından orijinal verinin bir yaklaşığı, değiştirilmiş katsayılarla yeniden kurulur.

Dalgacık analizi, bir görüntünün bilgisini yaklaşık (approximation) ve detay alt sinyallerine ayırır. Alt bantlar yukarı örnekleme (upsampling) ile birleştirilerek görüntü yeniden kurulur.

Dalgacık dönüşümünün avantajı şöyle anlatıldı: blok tabanlı dönüşümlerde görüntü genellikle 8x8 ya da 16x16 piksellik bloklara bölünür ve her blok ayrı dönüştürülüp iletilir. Dalgacık dönüşümü ise alt görüntülere değil bütün görüntüye uygulanır, bu yüzden bloklanma bozulması (blocking artifact) oluşmaz. Bloklanma bozulması, kayıplı sıkıştırmanın görüntü, ses ya da videoda yol açtığı bozulmadır. Bu, dalgacık sıkıştırmasının diğer dönüşüm tabanlı yöntemlere karşı başlıca üstünlüğüdür.

Bir görüntü, ayrık dönüşümün katsayılarıyla gösterilebilir. Bir milyon katsayı varsa bunların çoğu çok küçüktür ya da ihmal edilebilir; belki yalnız birkaç bini büyüktür. Bu yüzden küçük değerler sıfırlanarak görüntü kalitesinde belirgin bir değişiklik olmadan sıkıştırma yapılabilir.

2.3 Haar DWT adımları ve örnek

Slayttaki adımlar (ekrandan, 00:15:40):

1. Her örnek çiftinin ortalamasını bul.
2. Ortalama ile örnekler arasındaki farkı bul.
3. Dizinin ilk yarısını ortalamalarla doldur.
4. Normalize et.
5. Dizinin ikinci yarısını farklarla doldur.
6. İşlemi dizinin ilk yarısı üzerinde tekrarla.

Örnek: Sinyal $[1, 3, 5, 7]$ olsun.

- 1. iterasyon:
  - Ortalamalar: $(1+3)/2 = 2$, $(5+7)/2 = 6$.
  - Farklar (örnek eksi ortalama): $1 - 2 = -1$, $5 - 6 = -1$.
  - Dizi: $[2, 6, -1, -1]$.
- 2. iterasyon (yalnız ilk yarı, yani $[2, 6]$ işlenir):
  - Ortalama: $(2+6)/2 = 4$.

- Fark: $2 - 4 = -2$.
- Dizi: $[4, -2, -1, -1]$.

Slayttaki tablo bu iki satırı verir (python ile doğrulandı). Slaytta adımlar listesinde "Normalize" yer alır, ama örnekteki sayılarda normalize uygulanmamıştır. Slaydın sağında aynı sonuç dikdörtgen dalga biçimleriyle gösterilmiştir: 4 bütün sinyalin ortalamasıdır, -2, -1, -1 ise giderek daha ince ölçekteki detaylardır.

2.4 İki boyutlu ayrıştırma

İki boyutlu bir görüntüye Haar DWT uygulanırken önce görüntünün satırlarına tek boyutlu filtre uygulanır; L (alçak) ve H (yüksek) iki alt bant çıkar. Ardından sütunlara tek boyutlu filtre uygulanır ve dört alt bant elde edilir (ekrandan, 00:18:30):

- I_{LL}: görüntünün yaklaşık (kaba) sürümü.
- I_{HL}: yatay kenar bilgisi.
- I_{LH}: dikey kenar bilgisi.
- I_{HH}: çapraz kenar bilgisi.

İkinci seviyede LL bandı yeniden dört parçaya ayrılır (LL2, HL2, LH2, HH2); üçüncü seviyede LL2 ayrılır (LL3, HL3, LH3, HH3). Bir slaytta 512x512 bir görüntünün art arda ayrıştırılmasıyla yaklaşık görüntünün 256x256, 128x128, 64x64 ve 32x32 boyutlarına indiği görsel olarak gösterildi (ekrandan, 00:20:00).

Makalede yazarın uygulaması şu adımları içerir: görüntüyü oku, Haar dalgacığıyla iki boyutlu DWT uygula, orijinal görüntünün katsayılarını hesapla, görüntüyü ayrıştırıp göster ve eşiklemeyle sıkıştır.

2.5 Sonuçlar

Sonuç slaydı (ekrandan, 00:23:10):

Ölçüt	Değer
Seviyeye bağlı eşik	3,6
Yaklaşık katsayı sayısı	1
Yeniden oluşturma verimi	99,9865
Sıkıştırma skoru (yüzde)	60,8978

- Görüntü sıkıştırma için etkili ve verimli yöntemin dalgacık dönüşümü olduğu söylenmiştir.
- Önerilen yöntem görüntüyü hızlı sıkıştırır.
- 128x72 piksellik görüntü birinci seviyede 64x36, ikinci seviyede 32x18, üçüncü seviyede 16x9 olarak ayrıştırılmıştır.
- Yaklaşık %60 sıkıştırma oranı elde edilmiştir.
- Haar dalgacık dönüşümüyle hem kayıplı hem kayıpsız sıkıştırma yapılabilir.

2.6 Hocanın yorumu

Hoca, dalgacık dönüşümünün arkasında çok fazla matematik olduğunu, ama genelde hazır fonksiyonların alınıp kullanıldığını, özellikle Matlab'de kod yazılıyorsa böyle olduğunu söyledi. Makaledeki sıkıştırma skorunu düşük buldu: dalgacık dönüşümünün DCT'den daha iyi çalışması gerekir ve hoca kendisi kullandığında başarının daha yüksek olduğunu belirtti.

Sunan öğrenci düşük skorun eşik değerinden kaynaklanabileceğini ve yalnız 1 yaklaşık katsayı alınmasının yetersiz olduğunu, 2-3 katsayının denenmesi gerektiğini söyledi. Hoca buna, hiyerarşinin de çok önemli olduğunu ekledi: ayrıştırmanın kaç seviyeye kadar indiği sonucu etkiler, bu çalışmada 3. seviyede bırakılmıştır.

3. Sunum 2: Sinir Ağlarıyla Kayıpsız Veri Sıkıştırma

İkinci sunumun konusu, sinir ağları kullanılarak kayıpsız veri sıkıştırmadır. Amaç, LSTM (Long Short-Term Memory) ve Transformer tabanlı modellerle hızlı eğitilebilen bir kayıpsız sıkıştırıcı

gerçekleştirmektir.

3.1 Sinir ağları ve tahmine dayalı yaklaşım

Sinir ağı, insan beynindeki sinir ağına benzer şekilde birbirine bağlı düğümlerden oluşan bir hesaplama sistemidir. Ham verideki gizli örüntüleri ve ilişkileri tanıyabilir, veriyi kümeleyip sınıflandırabilir. Basit bir sinir ağı giriş katmanı, gizli katman ve çıkış katmanından oluşur.

Kayıpsız sıkıştırıcı geleneksel tahmine dayalı (predictive) yaklaşımı kullanır. Slayttaki adımlar (ekrandan, 00:28:50):

1. Her t anında kodlayıcı, s_0'dan s_{t-1}'e kadar bütün önceki sembolleri bilerek, sinir ağı modeliyle sonraki s_t sembolünün olasılık vektörü p'yi hesaplar.
2. Gerçek sembol s_t, aritmetik kodlayıcıyla yaklaşık $-\log_2(p_{s_t})$ bit kullanılarak kodlanır. Ardından model, s_t sembolü bilinerek güncellenir.
3. Kod çözücü simetrik çalışır, bu yüzden model parametrelerinin gönderilmesine gerek yoktur. Kodlayıcı ve kod çözücü modellerini aynı biçimde günceller.
4. Ön işlem yapılmazsa s_t, t konumundaki byte'tır. Bu durumda 0'dan $N_s - 1$'e kadar $N_s = 256$ farklı sembol değeri vardır.
5. Ön işlem yapılırsa her sembol bir byte dizisini temsil eder. Büyük modellerde yaklaşık $N_s = 16000$ sembollük bir sözlük kullanılır.

3.2 LSTM ve Transformer

Slayttaki tanıma göre LSTM, geri besleme bağlantıları olan yapay bir tekrarlayan sinir ağı (RNN) mimarisidir. Yalnız tek veri noktalarını (görüntü gibi) değil, bütün veri dizilerini (konuşma ya da video gibi) de işleyebilir. Yaygın bir LSTM birimi bir hücre, bir giriş kapısı, bir çıkış kapısı ve bir unutma kapısından oluşur. Hücre değerleri keyfi zaman aralıkları boyunca hatırlar, üç kapı da hücreye giren ve çıkan bilgi akışını düzenler (ekrandan, 00:30:00):

- Unutma kapısı ilgisiz bilgiyi unutur.
- Giriş kapısı yeni bilgi ekler ya da günceller.
- Çıkış kapısı güncellenmiş bilgiyi iletir.

Makaledeki LSTM modeli m katman LSTM hücresinden oluşur; her hücre, karşılık gelen hücrenin çıkışını girdi olarak alır. Hücre çıkışına bir normalizasyon işlemi eklenir; bu, verilerin -1 ile 1 arasına çekilmesi demektir.

İkinci model Transformer'dır. Sunumda dikkat başlıkları (attention head) ve geçmiş zaman adımları üzerinden hesaplanan denklemler gösterildi; sinüzoidal konum gömmeleri yerine öğrenilen göreli konum gömmeleri kullanıldığı söylendi. Büyük modelin bir tablosunda katman sayısı 6 ve model boyutu 512 gibi değerler, bellek olarak yaklaşık 348 MB ve 27,3 milyon parametre verildi.

Sonuçların her CPU ve işletim sisteminde aynı (deterministik) olması için harici bir kütüphane kullanılmıştır.

3.3 Ön işlem

Sıkıştırma oranını ve hızı artırmak için bir ön işlem adımı eklenmiştir. İki tür ön işlem vardır:

- **CMIX ön işlemcisi:** küçük LSTM modeliyle kullanılır. Anlamlı bir karşılaştırma için CMIX ve lstm-compress'in ön işlemcisi yeniden kullanılmıştır. Bu ön işlemci, kelimeleri önceden hazırlanmış bir sözlükten değiştirir.
- **Alt kelime (subword) tabanlı ön işlemci:** büyük modellerle kullanılır.

Alt kelime tabanlı ön işlemcinin slayttaki açıklaması (ekrandan, 00:36:45):

- Büyük modeller, her sembolün bir byte dizisini temsil ettiği bir ön işlemci kullanır.
- Girdi üzerinde ilk geçişte, CMIX ön işlemcisindeki gibi bütün büyük harfler kaçış (escape) kodlarıyla küçük harfe çevrilir.
- İkinci geçişte, iyi seçilmiş iki sembol birleştirilerek sözlüğe yinelemeli olarak yeni bir sembol eklenir. Bu fikir Byte Pair Encoding'den esinlenmiştir.

- Birleştirilecek semboller, sıfırıncı derece entropide beklenen azalmayı en büyük yapacak şekilde seçilir. Frekansı belli bir eşiğin altına düşen semboller çıkarılır. İkinci geçiş kelime ayırıcılarıyla ilgili bir varsayım yapmadığı için her dilde çalışır.

3.4 Sonuçlar

Slayttaki tablolar (ekrandan, 00:40:00). Tablo 2, enwik8 için sonuçlar:

Program ya da model	Sıkıştırılmış boyut (byte)	Oran (bpb)	Hız (kB/s)
gzip -9	36.445.248	2,92	17400
xz -9	24.865.244	1,99	1020
lstm-compress	20.494.577	1,64	4,4
CMIX (v17)	14.877.373	1,19	1,56
LSTM (small)	20.500.039	1,64	41,7
Transformer	18.126.936	1,45	1,79
LSTM (large1)	16.981.765	1,36	3,25
LSTM (large2)	16.791.077	1,34	2,38

Tablo 3, enwik9 için sonuçlar:

Program ya da model	Sıkıştırılmış boyut (byte)	Oran (bpb)
gzip -9	322.591.995	2,58
xz -9	197.331.816	1,58
CMIX (v17)	116.394.271	0,93
Transformer	130.440.948	1,04
LSTM (large1)	128.292.351	1,03
LSTM (large2)	125.623.896	1,00

Oran sütunu byte başına bit (bpb) cinsindendir. Sıkıştırılmış boyutun bit sayısı girdinin byte sayısına bölünerek bulunur; enwik8 10^8, enwik9 10^9 byte olduğu için örneğin lstm-compress için $20.494.577 \times 8 / 10^8 \approx 1{,}64$ olur (python ile tablodaki bütün oranlar bu şekilde doğrulandı).

Sunan öğrenciye göre makaledeki uygulama, benzer bir modelle lstm-compress'ten çok daha hızlıdır. Deneylerde Transformer modeli LSTM'den daha kötü sonuç vermiştir; LSTM daha yüksek sıkıştırma sağlar. Transformer, dil modelleme karşılaştırmalarında en iyi performansı verse de burada daha yavaştır. Bir şekil, büyük LSTM ve Transformer modelleri için sıkıştırma oranının dosya ofsetine göre değişimini, 5 MB üzerinden hareketli ortalama ile gösterir.

Sonuç olarak makalede LSTM tabanlı bir kayıpsız sıkıştırıcı sunulmuştur. Yavaş olsa da basittir ve bellek tüketimi makuldür. Tabloya bakıldığında sinir ağı modelleri gzip ve xz'den çok daha iyi oran verir, ama hızları gzip'in binlerce kat altındadır.

3.5 CMIX sorusu

Hoca, tablodaki CMIX'in ne olduğunu, başka birinin çalışmasına verilen bir ad olup olmadığını sordu. Sunan öğrenci soruyu cevaplayamadı ve hoca devam etmesini istedi (bkz. Örnek Sorular).

> **Ek bilgi (derste anlatılmadı):** CMIX büyük olasılıkla Byron Knoll'un geliştirdiği, çok sayıda modelin tahminlerini birleştiren (context mixing) bir kayıpsız sıkıştırma programıdır. PPM gibi bağlam tabanlı modellerin daha genel bir hâli olarak düşünülebilir. Tablodaki karşılaştırmada en iyi oranı veren, ama en yavaş programlardan biridir.

4. Sunum 3: FastText.zip, Metin Sınıflandırma Modellerinin Sıkıştırılması

Üçüncü sunumda bir öğrenci, metin sınıflandırma modellerinin sıkıştırılmasını ele alan FastText.zip makalesini Türkçe olarak anlattı. Makale özellikle sınıflandırma doğruluğu ile model boyutu arasındaki uzlaşmayı (trade-off) inceler.

4.1 Arka plan

Metin sınıflandırma, doğal dil işlemede önemli bir sorundur. Sinir ağlarına dayalı derin öğrenme teknikleri pek çok NLP uygulamasında en iyi sonuçlara ulaşmıştır; ama bazı metin sınıflandırma problemlerinde derin ağların önceki teknikleri ikna edici biçimde geçemediği gösterilmiştir.

Sunumda ilgili çalışmalar dört başlıkta özetlendi:

- **Metin sınıflandırma modelleri:** doğrusal sınıflandırıcılar ölçeklenebilir ve çoğu zaman en iyi performansı verir, ama kelime ve n-gram gömmelerinin saklanmasını gerektirdikleri için bellek açısından verimsizdir. N-gram, metnin 1'li, 2'li, 3'lü gruplara bölünmesidir.
- **Dil modellerinin sıkıştırılması:** klasik yaklaşımlar entropiye dayalı budama ve niceleme dir. Budama, modelde yalnız en önemli n-gramları tutup olasılığı belli bir eşiğin altında kalanları dışarıda bırakır.
- **Benzerlik tahmini ve arama için sıkıştırma:** vektörleri kısa kodlara sıkıştırmak için geniş bir literatür vardır. En popülerlerinden biri LSH'dir: rastgele izdüşümlere dayanan bir ikilileştirme tekniğidir ve iki vektör arasındaki kosinüs benzerliğini, ikili kodları arasındaki Hamming mesafesinin monoton bir fonksiyonuyla tahmin eder.
- **Sinir ağı sıkıştırma modelleri:** evrişimsel sinir ağları için vektör niceleme ve ağın ikilileştirilmesi gibi yöntemler kullanılmıştır. Bu çalışmalar, sınıflandırma modellerinin aşırı parametreli olduğu için kolayca sıkıştırılabildiğini gösterir.

4.2 Önerilen yaklaşım: metin sınıflandırması

Slayttaki açıklama (ekrandan, 00:50:00): metin sınıflandırmasında doğrusal sınıflandırıcılar daha karmaşık, daha derin modellerle rekabet eder ve çok daha hızlı eğitilir. Farklı sınıflar arasında bilgi paylaşırken hesap yükünü azaltmak için düşük sıralı (low-rank) bir kısıtlama kullanılır; bu, nadir sınıfların yalnız birkaç eğitim örneğine sahip olabildiği geniş çıktı alanlarında kullanışlıdır. Model, N belge üzerinde şu kaybı en aza indirir:

$$\sum_{n=1}^{N} \ell(y_n, BAx_n)$$

Burada x_n tek-sıcak (one-hot) vektörlerden oluşan bir torba, y_n ise n'inci belgenin etiketidir. Büyük bir kelime haznesi ve büyük bir çıktı alanı olduğunda A ve B matrisleri büyüktür ve gigabaytlarca bellek gerektirebilir.

4.3 Çarpım niceleme (PQ)

Çarpım niceleme, sıkıştırılmış alanda yaklaşık en yakın komşu araması için popüler bir yöntemdir. Gerçek değerli bir vektörü, kod çizelgesi (codebook) denilen önceden tanımlanmış, yapılandırılmış bir merkez kümesinden en yakın vektörü bularak yaklaştırır.

Sunum sırasında hoca PQ'nun açılımını sordu; sunan öğrenci önce bulamadı, sonra makaleden bakarak **Product Quantization** olduğunu söyledi. Hoca buradaki "product" kelimesinin "ürün" değil "çarpım" anlamına geldiğini, yani "çarpım niceleme" olduğunu belirtti.

Slayttaki özellikler (ekrandan, 00:51:10):

- Eğitim çok hızlıdır, çünkü alt niceleyicilerin az sayıda merkezi vardır: $b = 8$ için $2^8 = 256$ merkez.
- Test zamanında vektörler neredeyse hiç hesap ve bellek ek yükü olmadan yeniden kurulabilir.

- İkili kodlardan çok daha iyi performans verir; bu da onu görece sığ modelleri sıkıştırmak için doğal bir aday yapar. Son katmandan hemen önce PQ kullanılması, bir destek vektör makinesiyle birleştirildiğinde çok sınırlı bir doğruluk kaybına yol açar.
- Metin sınıflandırmasında vektörlerin normları önemlidir. Mutlak bir hata hedefi optimize edildiği için PQ düşük normlu bütün vektörleri sıfıra eşleyebilir ve kötü performans gösterir. Basit bir çözüm, vektörün normunu ve açısını (yönünü) ayırıp ayrı ayrı nicelemektir. Bu, performans kaybı olmadan nicelemeye izin verir, ama vektör başına fazladan bir bit gerektirir.

4.4 Bellek kullanımı ve budama

Bellek kullanımı büyük ölçüde kelime dağarcığının boyutuna bağlıdır. Dağarcığın büyük kısmı işe yaramaz ya da gereksiz olsa da dağarcığı doğrudan en sık kelimelere indirgemek tatmin edici değildir: en sık kullanılan kelimelerin çoğu (örneğin İngilizcede "the", "is" gibi) ayırt edici değildir. Sunan öğrenciye göre bu büyük kazanç PQ sıkıştırmasıyla tamamlayıcıdır ve iki strateji birlikte bazı veri kümelerinde model boyutunu 1000 kata kadar düşürür.

Dağarcık budama, genel performansı korumak için hangi kelime ya da n-gramların saklanacağını bulan bir özellik seçme problemidir. Önceden eğitilmiş bir modelden sabit K kelime ve n-gram alt kümesi seçilir. Bu, en yüksek normlu K kelime ve n-gramın seçilmesine indirgenebilir. Ama bazı belgeler en iyi K özellikten hiçbirini içermeyebilir ve performans çok düşebilir. Bu yüzden en iyi özellikler, bütün eğitim setini kapsamaları koşuluyla korunur.

Hashing hilesi ve Bloom filtresi: küçük modellerde sözlük hafızanın önemli bir kısmını alabilir. Bunu saklamak yerine hashing hilesi hem kelimelere hem n-gramlara genişletilir; bu, test zamanında neredeyse hiç ek yük olmadan yaklaşık 1-2 MB tasarruf sağlar. Hashing hilesiyle dağarcık budanırken kalan indekslerin bir listesi tutulmalıdır; test zamanında bu liste üzerinde ikili arama yapılır ($O(\log k)$) ve birkaç yüz KB'lik ek bellek gerekir. Bunun yerine Bloom filtreleri kullanmak test zamanındaki karmaşıklığı $O(1)$'e indirir ve birkaç yüz KB tasarruf sağlar, ama pratikte performansı biraz düşürür.

4.5 Deneyler

Model varsayılan ayarlarla FastText kullanılarak eğitilmiş, gömme boyutları ikinin kuvvetlerine ayarlanmıştır. Temel karşılaştırmalar LSH, PQ ve OPQ ile yapılmıştır.

- **Küçük veri kümeleri:** normalleştirmeden sonra PQ ve OPQ, gömme başına yalnız birkaç alt niceleyici kullanıldığında bile neredeyse kayıpsızdır. Gömme bileşenlerinin yarısını kullanmak ($k = d/2$) pratikte iyi sonuç vermiştir. PQ ile OPQ'nun normalleştirilmiş sürümleri arasındaki fark sınırlıdır; PQ daha hızlı eğitildiği için çalışmanın geri kalanında PQ kullanılmıştır.
- **Budama ve aşırı sıkıştırma:** normal PQ tam modele göre 10 kat sıkıştırma sağlar. 64 KiB ve 32 KiB'lık modellerde 1000 ile 4000 kat arasında sıkıştırma oranına rağmen performans düşüşü küçük kalmıştır.
- **Büyük veri kümesi:** 312.116 etiketli bir hashtag tahmin görevi ele alınmış; kelimelerin en az 10 kez geçmesi koşuluyla yaklaşık 1,5 milyon kelimelik bir sözlük kullanılmıştır. Tam model, eksik kelimeler yüzünden test setinin %11,6'sını kaçırır. 1 milyon özellikle yapılan saf budama, test setinin yaklaşık %30-40'ını kaçırır ve performansı önemli ölçüde düşürür. Eğitim setinde en yüksek kapsamayı gözeten budama ise test setinde kapsama kaybı yaşamaz ve daha küçük bir performans düşüşüne yol açar.

Gelecek çalışma olarak vektör boyutunun kelime frekansına göre ayarlanması, entropi ve norm budama ölçütlerinin birleştirilmesi ve az kullanışlı özelliklerin budanmak yerine daha küçük birimlere ayrılması önerilmiştir.

4.6 Sonuç

Sonuç slaydı (ekrandan, 01:00:00):

- Evrişimsel sinir ağlarına dayanan son çalışmalarla karşılaştırıldığında FastText.zip genellikle daha doğrudur, eğitimi birkaç kat daha az zaman alır ve belleğin küçük bir kısmını kullanır.

- PQ ile bellek tasarrufu: aşağıdan yukarıya PQ stratejisi, gözle görülür bir performans kaybı olmadan 10 kat sıkıştırma sağlar. Yeniden eğitim olmadan, veri kümesine ve ayara bağlı olarak doğrulukta %0,1 ile %0,5 arasında düşüş görülür.
- Slayta göre yaklaşım, karşılaştırma ölçütlerinde FastText'ten "iki kat daha az" bellek gerektirir ve doğruluk açısından yalnız biraz daha düşüktür.

  Ders sonrası not: Slayttaki "iki kat daha az bellek" ifadesi büyük olasılıkla makaledeki "en az iki büyüklük mertebesi (yaklaşık 100 kat) daha az bellek" ifadesinin çevirisinden kaynaklanır. Sunumun kendisinde 10 kat (PQ) ve 1000 kata kadar (PQ ile budama birlikte) sıkıştırma oranları geçtiği için "iki kat" bu sayılarla tutarlı değildir.

4.7 Hocanın soruları ve yorumu

Hoca sunan öğrenciye anlattıklarından ne anladığını sordu ve sunumun makaleyi okuyup tercüme etmek gibi kaldığını, kendisinin kafasında bir şey canlanmadığını söyledi. Hoca, 17 slaytlık bir sunumun bir iki cümleyle özetlenebilmesi gerektiğini belirtti. Ayrıca FastText'in kelime vektörlerinde kullanılan ve n-gramları kullanan bir algoritma olduğunu hatırlatıp sıkıştırma yaparken FastText'ten nasıl yararlanıldığının anlaşılmadığını söyledi.

Sunan öğrenci kendi anladığını şöyle özetledi: budama yapılırken, örneğin 10 kelimelik bir cümlede önemli olmadığı düşünülen kelimeler atılıyor, metin n-gramlara bölünüp inceleniyor; doğruluk zaman zaman düşse de performans açısından etkili sonuçlar alınmış. Veri setinin küçük ya da büyük olması ve hangi kelimelerin önemli olduğuna karar verilmesi de çok önemli.

Soru: Burada sıkıştırılan şeyler n-gramlar mı, en çok geçen n-gramlar mı sıkıştırılıyor? **Cevap:** Sunan öğrenci, budamanın çok kullanılan bazı kelimeleri atmaya çalıştığını söyledi; Türkçe düşünülürse "ve" bağlacının her geçtiği yerde atılması gibi.

Soru: Kelimeler atıldıysa geri dönüş nasıl olacak? Bu durumda sıkıştırma kayıplı mı olur? **Cevap:** Sunan öğrenciye göre makalede geri dönüşümle ilgili hiçbir bilgi yoktur.

> **Ek bilgi (derste anlatılmadı):** FastText.zip'te sıkıştırılan şey metin değil, eğitilmiş sınıflandırma modelinin kendisidir (gömme matrisleri ve sözlük). Bu yüzden atılan kelimelerin metne geri konması gerekmez; kayıp, modelin doğruluğundaki küçük düşüş olarak ortaya çıkar. Bu anlamda yöntem kayıplı bir model sıkıştırmasıdır.

5. Sunum 4: RLE, Aritmetik Kodlama, Punctured Elias ve Goldbach Kodlarının Karşılaştırılması

Dördüncü sunumda bir öğrenci, metin verisinin RLE, aritmetik kodlama, Punctured Elias kodu ve Goldbach kodu ile sıkıştırılmasını karşılaştıran bir makaleyi anlattı. Hoca makalenin 2017 yılına ait olduğunu söyledi.

5.1 Giriş: kayıplı ve kayıpsız sıkıştırma

Teknolojideki gelişmeyle dosya boyutları büyümüş, büyük depolama ihtiyacı ve dosyaların internet üzerinden gönderilmesi, indirilmesi ya da yüklenmesindeki güçlükler gibi sorunlar çıkmıştır. Sıkıştırma yöntemleri iki ana gruba ayrılır. Slayttaki ağaç ve tablo (ekrandan, 01:10:00):

- Kayıpsız yöntemler (metinler, programlar): RLE, aritmetik kodlama, Goldbach kodu.
- Kayıplı yöntemler (görüntüler, videolar): JPEG, MP3, MPEG.

Kayıpsız	Kayıplı
Fark edilmeyen veriyi atmaz.	Fark edilmeyen veriyi atar.
Dosya orijinal biçimine geri getirilebilir (metin, görüntü, ses).	Dosya orijinal biçimine geri getirilemez (görüntü, ses, video).
Verinin kalitesinden ödün vermez.	Verinin kalitesinden ödün verilir.
Örnek: RLE, Huffman, aritmetik kodlama.	Örnek: ayrık kosinüs dönüşümü, ayrık dalgacık dönüşümü, fraktal sıkıştırma.

Kayıpsız	Kayıplı
Kayıplıya göre daha az veri tutma kapasitesi.	Daha fazla veri tutma kapasitesi.

5.2 RLE

Slayttaki açıklama (ekrandan, 01:12:35):

- Algoritma, aynı karakter art arda dört ya da daha fazla kez (üçten fazla) geliyorsa veriyi üç karakterlik bir diziyle sıkıştırır.
- Bir d verisi girdi akışında art arda n kez (3'ten fazla) geçiyorsa, n tekrar $n@d$ ile değiştirilir.
- Yöntem, veri yalnız iki sembol kullanıyorsa daha da verimli olabilir.
- Sıkıştırma oranı = sıkıştırılmamış verinin uzunluğu / sıkıştırılmış verinin uzunluğu.

Sunumda RLE'nin, tekrarlanan örnekler içeren verinin boyutunu azaltan bir teknik olduğu, tekrarlanan örneklere "run" denildiği ve her run'ın genellikle iki byte ile kodlandığı söylendi: ilk byte örnek sayısını (run count), ikincisi örneğin kendisini verir.

Örnek 1: AAAAAABBBXXXXXT verisi 15 byte'tır. RLE ile 6A 3B 5X 1T olarak, 8 byte ile kodlanır. Sıkıştırma oranı $15/8 = 1{,}875$'tir (ara adım eklendi; python ile doğrulandı).

Örnek 2: Slayttaki BBBBBBBBBAAAAAAAAAAAAAAAANMMMMMMMMMM verisi B09A16N01M10 olarak sıkıştırılmıştır: 9 B, 16 A, 1 N, 10 M.

> **Ders sonrası not:** Slayttaki kural "üçten fazla tekrar" der, ama örnekte 3 tekrarlı B'ler (3B) ve tek T (1T) de sayı-karakter çifti olarak yazılmıştır. Örnekler, her run'ın (sayı, karakter) çiftiyle yazıldığı basit RLE biçimine uyar; $n@d$ kuralı ise yalnız uzun tekrarları kodlayan biçimdir.

5.3 Aritmetik kodlama

Aritmetik kodlamada mesaj $[0, 1)$ aralığında bir sayıyla gösterilir; her sembolle geçerli aralık, sembolün olasılığına göre daraltılır. Sunan öğrenci, sayı çok büyüdüğünde kodlayıcı ya da kod çözücü cihazda özel olanaklar gerektiğini söyledi. Aralığın bölünmesi bir şekille gösterildi.

5.4 Punctured Elias kodu

Slayttaki açıklama (ekrandan, 01:17:30):

- Kod Peter Fenwick tarafından tasarlanmıştır.
- "Punctured" adı Error Code Memory (ECC) kavramından gelir. ECC, orijinal veriye ek olarak kontrol bitlerinden oluşan bir sayı dizisinden oluşur.

Örnek adımları:

1. n'in ikilik değeri yazılır ve bitleri ters çevrilir; böylece en sağdaki bit 1 olur. Ardından n'in bitleri için bayraklar önüne eklenir.
2. n'in her biti için 0 bayrağı oluşturulur; yalnız son (en sağdaki, 1 olan) bitin bayrağı 1'dir.
3. Bayraklar ters çevrilmiş n'in önüne eklenir ve en sağdaki bit atılır.

Örnek: $13 = 1101$. Ters çevrilince 1011 olur. Dört bayrak 0001 önüne eklenir ve 1011'in en sağdaki biti atılır. Slayttaki son kod: $0001|101$.

> **Ders sonrası not:** Slayt sonucu "gamma code" diye adlandırır. Elde edilen 0001101 dizisi gerçekten de 13'ün Elias gamma kodudur (3 sıfır, ardından 1101; python ile doğrulandı). Fenwick'in punctured Elias kodları (P1, P2) büyük olasılıkla bundan farklı olarak 1 bitlerinin sayısına göre bayrak kullanır; slayttaki anlatım iki kodu karıştırıyor olabilir.

Sunum sırasında sunan öğrenci çok heyecanlandığını söyledi; hoca bunların zaten bilinen konular olduğunu, heyecanlanmaya gerek olmadığını söyledi.

5.5 Goldbach kodları

Goldbach kodu, Goldbach sanısını (“2’den büyük bütün pozitif çift sayılar iki asal sayının toplamıdır”) kullanan bir algoritmadır. Üç kodu vardır (ekrandan, 01:20:00):

- **G0:** pozitif tamsayı n, $2(n+3)$ ile bir çift sayıya çevrilir ve bu sayıyı veren asal sayıların toplamı ters yazılır.
- **G1:** toplamı n’i veren iki asal P_i ve P_j ($i \leq j$) bulunur ve $(i, j-i+1)$ çifti gamma kodlarıyla kodlanır.
- **Goldbach kodları (G2):** G1’in bazı özel durumlarla genişletilmiş hâlidir.

G2’nin özel durumları (ekrandan, 01:20:40):

1. 1 ve 2 tamsayıları 110 ve 111 olarak kodlanır. 1 ve 2 dışında hiçbir kod 11 ile başlamaz.
2. Çift tamsayılar G1’deki gibi kodlanır, bir istisnayla: $n = P_i + P_j$ bulunursa, $(i, j-i+1)$ yerine $(i+1, j-i+1)$ çifti ile kodlanır. Böylece $i = 1$ ise 2’nin gamma kodu olan 010 yazılır. Bu, G2’de çift sayıların kodunun 1 ile başlamamasını ve her zaman $0\ldots$ biçiminde olmasını sağlar.
3. n bir P_i asal sayısıysa, $(i+1)$’in gamma kodu ve ardından tek bir 1 yazılır; kod $0\ldots1$ biçimindedir.
4. n tek bir sayıysa, G2 kodu tek bir 1 ile başlar ve ardından çift sayı $n-1$’in G2 kodu gelir; kod $1{:}0\ldots$ biçimindedir.

   Ek bilgi (derste anlatılmadı): Gamma kodu, derste ilk haftalarda görülen Elias gamma kodudur: n’in ikilik gösterimi L bit ise önüne $L-1$ tane 0 yazılır. Örneğin 2 = 10 olduğu için gamma kodu 010’dur. Goldbach kodlarında asal sayıların kendisi değil, asal dizisindeki sıra numaraları (i ve $j-i+1$) kodlandığı için küçük sayılar elde edilir.

5.6 Deney sonuçları

Slayttaki üç deney (ekrandan, 01:23:40):

Girdi	RLE	Aritmetik	Punctured Elias	Goldbach
5 karakter = 40 bit	80 bit	32 bit	64 bit	14 bit
10 karakter = 80 bit	64 bit	56 bit	75 bit	28 bit
15 karakter = 120 bit	240 bit	65 bit	88 bit	49 bit

Bu tabloya göre en kısa çıktıyı her üç deneyde de Goldbach kodu vermiştir. RLE, 40 ve 120 bitlik girdilerde çıktıyı iki katına çıkarmıştır; yani tekrar içermeyen bir metinde RLE sıkıştırma değil genişletme yapar. Sunumda deney girdilerinin içeriği ve sıkıştırma oranı grafikleri ayrıntılı açıklanmadı.

5.7 Hocanın yorumu

Hoca, makalede Elias kodları ve Goldbach kodlarının bulunduğunu, sunumun büyük ölçüde kendisinin ilk haftalarda anlattıklarını özetlediğini söyledi. Goldbach kodlarını ise derste vermediğini belirtti.

6. Sunum 5: Doğal Dil İşleme için CNN Sıkıştırma: Niceleme ve Budama

Beşinci sunumda bir öğrenci, doğal dil işlemede kullanılan evrişimsel sinir ağlarının (CNN) niceleme ve budama ile sıkıştırılmasını inceleyen bir makaleyi Türkçe anlattı.

6.1 Amaç ve CNN

Doğal dil işleme uygulamalarında derin öğrenme yaygın kullanılır; evrişimsel sinir ağları artık görüntü ve video işlemede de kullanılır. Makine çevirisi ve sesli yazma gibi uygulamaların sayısı ve mobil cihazlarda kullanımı her gün artmaktadır. Bu da derin ağlarda optimizasyon ihtiyacını doğurur.

Çalışmanın temel amacı, FPGA (Field Programmable Gate Array) üzerindeki gömülü sinir ağı hesaplamalarında niceleme (quantization) ve budama (pruning) tekniklerinin uygulanabilirliğini ve verimini incelemektir. Sunan öğrenci FPGA'yı sonradan programlanabilen bir kapı sistemi olarak tanımladı.

Slayttaki CNN bilgileri (ekrandan, 01:30:00):

- CNN'ler farklı ağırlık (katsayı) ve bias değerleri olan nöronların birleşmesiyle oluşur. Bu nöronlar çok boyutlu vektörleri girdi olarak alabilir.
- Evrişim (convolutional), doğrusal olmama (non-linearity), havuzlama (pooling, downsampling), düzleştirme (flattening) ve tam bağlı (fully-connected) gibi farklı katmanlar bulunur.
- CNN'lerde nicelemenin rolü karmaşıklığı, budamanın rolü ise sıkıştırma sayesinde işlem sayısını azaltmaktır.

6.2 Sayı gösterimi ve niceleme

Nicelemeye geçmeden önce sunan öğrenci kayan noktalı sayı standardını hatırlattı: 32 bitlik tek duyarlıkta (single precision) 1 bit işaret, 8 bit üs ve 23 bit mantis (anlamlı kısım) bulunur. Aynı yapı 64 bitlik çift duyarlıkta da vardır.

Bundan farklı olarak dinamik sabit nokta (dynamic fixed point) yaklaşımı vardır. Örneğin 16 bitlik bir gösterimde kesirli kısım için 10 bit seçilirse, tamsayı kısmına 5 bit kalır ve 1 bit de işaret bitidir. Slayttaki ilişki (ekrandan, 01:31:30):

$$frac_bits = total_bits - int_bits - 1$$

Örnekte $10 = 16 - 5 - 1$'dir (ara adım eklendi).

Slayttaki tanıma göre niceleme işleminde yoğun ve sürekli kümeler, daha seyrek ve ayrık kümeler hâlinde kısıtlanır; böylece ağ girdileri düzenlenmiş olur. Temelde bir aralık vardır ve bu aralığı temsil edecek sayılar seçilir. İki yöntem verildi:

- **Asimetrik tamsayı niceleme:**

$$q_{fxp} = Q(x_{flp}) = \mu + \sigma \cdot \text{round}(\sigma^{-1} \cdot (x - \mu))$$

Makaledeki durumda $\mu = 0$ ve $\sigma = 2^{-frac_bits}$ alınır; burada $int_bits = \lceil \log_2(\max_{x \in S} |x|) \rceil$'dir.

- **Simetrik tamsayı niceleme:**

$$q_{int} = Q(x_{flp}) = \text{ceil}\big((x - \mu)/((\max(X) - \min(X)) \cdot \sigma^{-1})\big)$$

Sunan öğrenciye göre çalışmada simetrik niceleme kullanılmıştır. Burada μ ortalama değerdir (genelde sıfır alınabilir), max ve min aralığın en büyük ve en küçük değerleridir; formül, değeri nicelenmiş bir tamsayıya çevirir.

6.3 Budama

Budama için derin öğrenme sonunda çıkan ağırlıkların bir dağılımı gösterildi. Örnekte 0,02 eşik değeri için budama yapılır: bu ağırlık değerlerinin toplam hesaba etkisi azdır. Sunan öğrenci bunu şöyle açıkladı: bir sayı 0,02 ile çarpılınca sonuca çok bir etkisi olmaz. Bu değerler budanarak ağ küçültülür.

6.4 Veri setleri ve model

Çalışmada 7 veri seti kullanılmıştır:

- MR (Movie Reviews)
- SST-1 ve SST-2 (Stanford Sentiment Treebank)
- Subj (Subjectivity)

- TREC (Text Retrieval Conference)
- CR
- MPQA (Opinion Corpus)

Makine öğrenmesinde bazı veri setlerinde çapraz doğrulama (örneğin 10 katlı) yapılmış, diğerleri eğitim, test ve doğrulama olarak bölünmüştür.

Mimari, daha önce Yoon Kim'in cümle sınıflandırma çalışmasında kullanılan CNN-static ağıdır. Bu ağ, duygu analizi ve cümle sınıflandırmada MR üzerinde yaklaşık %81 doğruluk verir. Model 5 katmanlıdır: önce kelime gömme (sunan öğrenciye göre 300 boyutlu GloVe vektörleri), ardından 2 evrişim katmanı ve 2 yoğun (dense) katman.

6.5 Deneyler

Duyarlılık azaltma iki yerde yapılır: katmanların ağırlıkları ve aktivasyon fonksiyonlarının çıktıları. Ağırlıklar için öğrenilen değerlerin minimum ve maksimumu alınır; budama ve niceleme sonra sistem tekrar çalıştırılır, aralığın dışında kalanlar atılır. Aktivasyon çıktıları için belli aralıklarla kutular (bin) oluşturulur; örneğin 16 bitlik bir duyarlılık düşünülüyorsa 2^{16} kutu oluşturulur ve değerler oralarda tutulur.

Sıkıştırılabilirliği test etmek için üç tür deney yapılmıştır:

1. **Birinci tür:** duyarlılık azaltma 8 farklı yerde yapılır (5 katmanın ağırlıkları ve aktivasyonları: Embedding, Convolution 1, Convolution 2, Dense 1, Dense 2). Bir yerin değeri azaltılırken diğerleri sabit tutulur. Örneğin 32 bitten 16 bite inildiğinde doğrulukta değişiklik olmaz; 8, 7 diye inildikçe değişim gözlenir. Aynı katmanın ağırlık ve aktivasyonu için aynı bit değeri kullanılmıştır.
2. **İkinci tür:** kısıt olmadan farklı bit değerleri denenir. 8 noktada 32'den 1'e kadar 10 farklı değer için bu 10^8 deney demektir. Süre çok uzun olduğu için rastgele tepe tırmanma (random hill climbing) ile arama yapılmış ve 50 kez tekrarlanmıştır.
3. **Üçüncü tür:** niceleme ve budama farklı değerlerle birlikte uygulanmış ve MR veri seti üzerindeki sonuçlar incelenmiştir.

Önceden bir doğruluk kaybı eşiği belirlenir (örneğin %1'den fazla azalmasın) ve her veri seti için bu eşiği aşmayan en düşük bit değerleri aranır. Örneğin en uygun değer, gömme için 5 bitlik duyarlılık, evrişim aktivasyonları için 5 bit gibi bulunmuştur.

Hoca, SST-1 veri setindeki başarının oldukça düşük olduğunu, bunun verinin içeriğiyle ilgili olabileceğini söyledi. Sunan öğrenci, cümle sınıflandırmada kullanılan bütün yöntemlerde bu veri setinde %45-48 civarında bir başarı olduğunu, önceki çalışmalarda da benzer değerler bulunduğunu, bunun veri setiyle ilgili olduğunu düşündüğünü söyledi.

> **Ek bilgi (derste anlatılmadı):** SST-1, cümleleri beş sınıfa (çok olumsuz, olumsuz, nötr, olumlu, çok olumlu) ayıran ince taneli bir duygu veri setidir; SST-2 ise yalnız olumlu ve olumsuz iki sınıf içerir. Beş sınıflı problemde rastgele tahminin başarısı %20 olduğundan %45-48 civarı doğruluklar bu veri seti için olağandır; SST-2'de doğruluk çok daha yüksektir.

6.6 Sonuçlar

Son set deneylerin grafiklerinde (ekrandan, 01:40:00) bir eksende niceleme (32 bitten 1 bite), diğer eksende budama eşiği (0,01'den 0,15'e) vardır:

- Soldaki grafik, budamalı hesap sayısının budamasız hesap sayısına oranını (calculation ratio) gösterir. Budama eşiği düşük tutulduğunda oranlar neredeyse 1'dir; budama arttıkça hesap oranı düşer. Belli bir eşikten sonra ağırlık değerlerinin tamamı budandığı için artık hesap yapılmaz.
- Sağdaki grafik doğruluğu gösterir. Küçük budama değerlerinde ve duyarlılık çok düşürülmeden MR veri seti için %80 civarında doğruluk elde edilir. Budama arttıkça ve duyarlılık azaldıkça bir süre sonra doğruluk düşer.

Sunan öğrenciye göre bütün derin ağda yaklaşık %84'lük bir küçülme elde edilmiştir. Son tabloda FPGA'da farklı budama ve duyarlılık değerleri için kullanılan flip-flop ve lookup table

sayıları verilmiştir: duyarlılığın değişmesi kullanılan kaynakları azaltır; budama ve duyarlılığın birlikte kullanılması bu sayıyı daha da azaltır.

Sunumdan sonra hoca, birinden dinleyince konuya çok hâkim olunamadığını, bazı makalelerin dilinin de içeriğinin de ağır olduğunu, bazı şeyleri bilmeyi gerektirdiğini söyledi. Okuyan kişi kendisi de tam bilmiyorsa okuduğunu anlatır; bu yüzden makale seçmek çok önemlidir.

7. Sunum 6: Dinamik Değişkenden Değişkene Sıkıştırma (D-v2v)

Son sunumda bir öğrenci, dinamik değişkenden değişkene (dynamic variable-to-variable) bir sıkıştırma yöntemini anlatan bir makaleyi inceledi. Makalede yöntem kısaca D-v2v olarak adlandırılır.

7.1 Amaç ve genel yapı

Makalenin amaçları:

- Değişken uzunlukta girdi sembollerini toplayan bir modelleyici ve daha sık modellere daha kısa kod sözcükleri atayan değişken uzunluklu istatistiksel bir kodlayıcı kullanmak.
- İngilizce metinlerde p7zip gibi en güçlü sıkıştırıcılarla rekabet edebilecek sıkıştırma oranlarına ulaşmak.

Yöntem dinamik, tek geçişli bir sıkıştırıcıdır. Dynamic End-Tagged Dense Code (DETDC) yönteminin bir versiyonu olarak geliştirilmiş, onu temel almıştır. Üç adımdan oluşur: ayrıştırma (parsing), kodlama ve kod çözme (alma). Sıkıştırma ve çözme sırasında sorun olmaması için gönderici ve alıcı tarafındaki bilgiler birbiriyle senkron olmalıdır.

7.2 Ayrıştırma

Ayrıştırmada giriş metni taranır ve değişken sayıda kelimeden oluşan dizileri temsil eden semboller oluşturulur. Semboller ikiye ayrılır:

- **Terminal semboller:** yalnız bir kelimeden oluşur. Sembol listesinde olmayan yeni bir kelime geldiğinde oluşturulur.
- **Terminal olmayan semboller:** en az iki kelimeden oluşur. Bu kelimeler terminal ya da terminal olmayan sembollerden gelebilir.

Her seferinde tek kelime okunur. Okunan kelime sembol listesinde yoksa, yeni kelimeyi temsil eden bir terminal sembol S_i oluşturulur. Ayrıca bu sembol bir önceki gönderilen sembolle birleştirilip yeni bir terminal olmayan sembol S_{i+1} oluşturulur. Böylece listeye iki sembol eklenmiş olur.

Okunan kelime sembol listesindeki sembollerden birinin öneki ise, “okuma sırası” (reading sequence, RS) adlı listede saklanır. Kelimeler, okuma sırası listede bilinmeyen bir dizi hâline gelene kadar eklenmeye devam eder. Bilinmeyen bir dizi elde edildiğinde, okuma sırasının bilinen en uzun önekine karşılık gelen sembol alıcıya gönderilir. Gönderilen sembolü ve bir öncekini içeren yeni bir terminal olmayan sembol oluşturulup listeye eklenir. Bütün semboller “bilinen diziler” (known sequences) adlı listede saklanır.

7.3 Ayrıştırma örneği

Slayttaki örnek (ekrandan, 01:50:00): gönderilecek mesaj “the more I know about you the more I know about me” olsun.

1. İlk aşamada bilinen diziler listesi boştur. İlk okunan “the” kelimesi ilk sembol S_0 olarak eklenir ve alıcıya gönderilir.
2. “more” okunduğunda, sembol listesinde bir önekte bulunmadığı için yeni bir kelimedir. “more” terminal sembol S_1 olarak, “the more” terminal olmayan sembol S_2 olarak eklenir.
3. Aynı şekilde devam edilir. “you” kelimesine gelindiğinde bilinenler listesi şöyledir:

Sembol	Dizi
S_0	“the”

Sembol	Dizi
S_1	“more”
S_2	“the more”
S_3	“I”
S_4	“more I”
S_5	“know”
S_6	“I know”
S_7	“about”
S_8	“know about”
S_9	“you”
S_{10}	“about you”

4. Bundan sonra “the” okunur. Sembol listesinde olduğu için (S_0) okuma sırasına eklenir: RS = “the”.
5. “more” okunur. “the more” listede olduğu için (S_2) okuma sırası güncellenir: RS = “the more”.
6. “I” okunur. RS = “the more I” listede olmadığı için en son eşleşen sembol S_2 alıcıya gönderilir. $S_{11} = S_9 \| S_2$, yani “you the more”, yeni sembol olarak listeye eklenir.

S_{11}’in S_9 ile birleştirilmesinin nedeni, alıcıya bir önceki adımda gönderilen sembolün S_9 (“you”) olmasıdır.

7.4 Kodlama ve kod çizelgesi

Semboller alıcıya gönderilmeden önce kodlanmalıdır. Bir sembolün kodu yalnız, kullanım sıklığına göre sıralanmış listedeki sırasıyla belirlenir. Bir sembol gönderildiğinde frekansı artar, böylece ona atanan kod sözcüğünün uzunluğu azalabilir; yöntem en sık kullanılan sembollere en kısa kod sözcüklerini atar. Terminal sembol ilk kez gönderilirken düz biçimiyle gönderilir: hem kod sözcüğü hem kelimenin açık hâli gider. Daha önce kod çözücünün gördüğü bir sembolse yalnız kod sözcüğü gönderilir.

Slayttaki açıklama (ekrandan, 01:52:20): sembolleri kodlamak için gereken bütün bilgi kod çizelgesi (codebook) adlı yapıda tutulur. Her kayıt bir sembole karşılık gelir ve $\langle l, r, freq, voc \rangle$ biçiminde saklanır:

- l ve r sembolün dizisini temsil eder. Sembol terminal ise l kelimenin kendisini tutar, r’ye -1 atanır. Terminal değilse l ve r, sembolü oluşturan sembollerin indisleridir.
- $freq$ sembolün sıklığını tutar; terminal olmayan semboller oluşturulurken frekansları 0 atanır.
- voc sembolün sıklığa göre sıralanmış sözcük içindeki konumunu tutar.

Slayttaki “you” eklendiği andaki kod çizelgesi:

İndis	l	r	$freq$	voc
0	the	-1	1	0
1	more	-1	1	1
2	0	1	0	9
3	I	-1	1	2
4	1	3	0	7
5	know	-1	1	3
6	3	5	0	6
7	about	-1	1	4
8	5	7	0	8
9	you	-1	1	5
10	7	9	0	10

Örneğin 2. kayıt “the more”dur: $l = 0$ (“the”), $r = 1$ (“more”). Slaytta ayrıca semboller bir trie (ağaç) yapısında gösterilmiştir.

7.5 pos ve top dizileri

Sıklığa göre sıralama için pos ve top olmak üzere iki dizi kullanılır (ekrandan, 01:55:30):

- **pos:** sembolleri sıklığa göre azalan sırada tutar. $pos[i] = j$, en sık kullanılan i'inci sembolün kod çizelgesinin j'inci indisinde olduğunu söyler.
- **top:** her olası frekans değeri f için $top[f] = x$, bu frekansa sahip ilk sembolün kod çizelgesindeki indisinin $pos[x]$'te olduğunu söyler.

"you" eklendiği anda:

- pos $= [0, 1, 3, 5, 7, 9, 6, 4, 8, 2, 10]$ (konum 0'dan 10'a).
- top[0] $= 6$, top[1] $= 0$.

Yani frekansı 1 olan semboller (0, 1, 3, 5, 7, 9) pos'un 0-5 konumlarında, frekansı 0 olanlar 6. konumdan itibaren durur.

Güncelleme örneği: sonraki adımda S_2 "the more" kodlanıp alıcıya gönderilir.

1. Kod çizelgesinin 2. indisinin frekansı 1 artırılır (0'dan 1'e).
2. top dizisinde frekansı 0 olan sembollerin başlangıç konumu 6'dır. Frekans arttığı için hem pos hem top güncellenmelidir.
3. voc[2] ve voc[6] yer değiştirir: voc[2] 9'dan 6'ya, voc[6] 6'dan 9'a geçer.
4. voc değiştiği için pos dizisi $pos[6] = 2$, $pos[9] = 6$ olarak güncellenir.
5. top[0] $= 7$ olur: frekansı 0 olan semboller artık 7. konumdan başlar.

7.6 Kod çözme

Kod çözme için yalnız bir kod çizelgesi ve yardımcı bir tablo gerekir. Kod çözücünün kod çizelgesi offset, uzunluk ve frekans alanlarından oluşur. Kod çözme adımında ya bilinen bir sembole karşılık gelen bir kod çözülür ya da bir kaçış (escape) kodu gelir ve ardından kelime düz biçimde gönderilir.

Bir sembolün kodu çözüldükten sonra, kod çizelgesini göndericiyle senkron tutmak için son iki çözülen sembolden oluşan yeni bir terminal olmayan kayıt eklenir. Bu, alıcının gönderen tarafından işlenen modelin aynısını kurmasını ve orijinal metni elde etmesini sağlar. Örneğin "the" ve "more" çözüldükten sonra "the more" kaydı eklenir.

Her yeni sembol oluşturulduğunda, sembolün açılmış metin içindeki ilk geçtiği konum offset değeri olarak kullanılır; uzunluk ve frekans da kaydedilir. Örneğin "the"nin offset'i 0, uzunluğu 3'tür. Gönderici "I know" dizisinin kod sözcüğünü ilettiğinde alıcı, daha önce çözülmüş metindeki offset ve uzunluk bilgisini kullanarak diziyi açılmış metinden kopyalar, frekansı artırır ve indisleri kodlayıcıdaki gibi günceller.

7.7 Analiz

Sıkıştırma oranı tablosu (ekrandan, 02:00:00), sıkıştırılmış boyutun orijinale oranını yüzde olarak verir:

Veri	Detdc	v2vdc (lzma words)	v2vdc$_H$ (lzma words)	v2vdc (plain words)	v2vdc$_H$ (plain words)	D-v2v	Repair +sHuff	lzma def	lzma -9 -e	p7zip	bzip2 def	Boyut (KB)
FT91	35,64	27,15	26,65	30,11	29,61	28,60	24,00	25,50	25,25	25,52	27,06	14.404
CR	31,99	23,55	23,13	24,73	24,31	22,86	20,16	22,05	20,83	21,63	24,14	49.888
ZIFF	33,79	24,01	23,60	24,66	24,25	23,14	20,33	23,40	21,64	22,98	25,10	180.879
ALL	33,66	22,81	–	23,39	–	22,67	–	23,23	21,34	22,80	25,98	1.055.391
WL	15,06	4,13	–	4,44	–	2,90	1,43	1,30	1,11	1,39	6,94	45.867
ENG	35,21	–	–	–	–	5,52	2,17	0,55	0,55	0,55	3,73	102.400
EINS	30,14	0,97	–	0,98	–	0,27	0,07	0,07	0,07	0,07	5,17	456.667

Sunan öğrenci, tablodaki yüzdeleri ilk okuduğunda karıştırdığını söyledi. Hoca bunu şöyle açıkladı: buradaki değer başarım değil orandır. %35, 100 birimin 35 birime düştüğü anlamına gelir; yani %65'lik bir kazanç vardır. Bu yüzden tabloda küçük değer daha iyidir.

Slayttaki yorumlar:

- D-v2v bütün veri kümelerinde v2vdc ve v2vdcH sonuçlarını yakalayabilmiş ve iyileştirebilmiştir.
- D-v2v, orijinal DETDC'yi normal İngilizce veri setlerinde yüzde 10'dan fazla puanla aşmıştır; tekrarlayan koleksiyonlarda (WL, ENG, EINS) DETDC'yi tamamen geride bırakır.
- Normal metinlerde D-v2v ve p7zip en büyük veri setinde (FT91, CR ve ZIFF birleştirilerek oluşturulan ALL) benzer değerler elde etmiştir. Diğer veri setlerinde kelime yerine karakterden yararlanılması p7zip, lzma ve Re-Pair'e avantaj sağlamıştır. Tekrarlayan metin koleksiyonlarında karakter düzeyindeki tekrar kelime düzeyinden yüksektir; ayrıca kelimelerin düz biçimde gönderilmesi D-v2v'yi olumsuz etkilemiştir. Pratikte D-v2v'nin sıkıştırması iyi olsa da genellikle Re-Pair, p7zip ve lzma'yı yakalayamamıştır.

Süre açısından hem sıkıştırma hem çözme süreleri karşılaştırılmıştır: D-v2v, p7zip, lzma ve Re-Pair'den hızlıdır; v2vdc ile aynı düzeydedir ve bzip2'den yavaştır. Temel alınan DETDC, alt dizilerin tespitiyle uğraşmadığı için çok daha basit ve hızlıdır. Çözmede, tekrarlamayan İngilizce metinlerde temel alınan yöntem ve v2vdc dışında D-v2v en hızlı tekniktir.

Bellek açısından, özellikle sıkıştırmada diğer yöntemlerle çok fark vardır: kod çizelgesi, dizi yapıları ve alt diziler gibi çok sayıda yapı kullanıldığı için bellek ihtiyacı yüksektir. Çözmede yalnız kod çizelgesi kullanıldığı için ihtiyaç daha makuldür, ama yine de diğer yöntemlerden fazladır.

7.8 Sonuç ve gelecek çalışmalar

Yöntemin İngilizce metinlerde rekabetçi sıkıştırma oranları elde ettiği ve hem sıkıştırmada hem özellikle çözmede hızlı olduğu gösterilmiştir. Alt dizileri işlemek için kullanılan trie yapısı sıkıştırmada çok bellek ister; bu yöntemin dezavantajıdır. Gelecek çalışma olarak bellek gereksinimini azaltmak için uygulamanın iyileştirilmesi ve alıcı tarafındaki işi ve bellek kullanımını azaltmak için Dynamic Lightweight Text Compression'daki fikirlerle asimetrik, hafif bir sürüm oluşturulması planlanmıştır.

Hoca, sunan öğrenciye çalışmaya çok hâkim olduğu ve emek harcadığı için teşekkür etti; makaledeki yöntemi "bizim yöntem" diye sahiplenecek kadar iyi anlattığını söyledi.

Soru: Bu yöntemi sözlük tabanlı bir sıkıştırmaya dönüştürmek mümkün mü? **Cevap:** Mümkündür; yöntem zaten kelime bazlı işlediği için.

Örnek Sorular

Soru 1 (ekrandan, 00:15:40). $[1, 3, 5, 7]$ sinyaline Haar DWT uygulayın. İki iterasyon sonundaki diziyi bulun.

Çözüm (derste verildi): 1. iterasyon: ortalamalar $(1+3)/2 = 2$, $(5+7)/2 = 6$; farklar $1-2 = -1$, $5-6 = -1$; dizi $[2, 6, -1, -1]$. 2. iterasyon: $(2+6)/2 = 4$, $2-4 = -2$; dizi $[4, -2, -1, -1]$ (python ile doğrulandı).

Soru 2 (derste sözlü, 00:24:11). Makalede Haar DWT ile elde edilen sıkıştırma skoru neden düşük olabilir?

Çözüm (derste verildi): hocaya göre dalgacık dönüşümü DCT'den daha iyi çalışmalıdır. Sunan öğrenci eşik değerinin ve yalnız 1 yaklaşık katsayı alınmasının (2-3 denenmeliydi) etkili olabileceğini söyledi; hoca ayrıştırmanın kaç seviyeye indiğinin (bu çalışmada 3) de önemli olduğunu ekledi.

Soru 3 (ekrandan, 00:40:00). enwik8 tablosunda lstm-compress 20.494.577 byte'a sıkıştırmıştır. Oran (bpb) nedir?

Çözüm (derste verilmedi, tablodaki değerden doğrulandı): $20.494.577 \times 8/10^8 \approx 1{,}64$ bpb. Tablodaki diğer oranlar da aynı şekilde bulunur (python ile doğrulandı).

Soru 4 (derste sözlü, 00:39:43). Tablodaki CMIX nedir?

Çözüm (derste verilmedi): Derste cevaplanmadı. CMIX büyük olasılıkla Byron Knoll'un geliştirdiği, birçok modelin tahminlerini birleştiren (context mixing) bir kayıpsız sıkıştırma programıdır. Tabloda enwik8'de 1,19, enwik9'da 0,93 bpb ile en iyi oranı verir.

Soru 5 (derste sözlü, 00:51:20). FastText.zip'teki PQ'nun açılımı nedir?

Çözüm (derste verildi): Product Quantization; hocaya göre "product" burada "çarpım" anlamındadır, yani çarpım niceleme. $b = 8$ için alt niceleyicinin $2^8 = 256$ merkezi vardır.

Soru 6 (derste sözlü, 01:05:06). FastText.zip'te sık kullanılan kelimeler atılıyorsa geri dönüş nasıl olur; sıkıştırma kayıplı mıdır?

Çözüm (derste verildi): sunan öğrenciye göre makalede geri dönüşümle ilgili bilgi yoktur.

Çözüm (derste verilmedi): Yöntem metni değil sınıflandırma modelini sıkıştırır; metnin geri kurulması gerekmez. Kayıp, modelin doğruluğundaki küçük düşüş (%0,1-0,5) olarak ortaya çıkar.

Soru 7 (ekrandan, 01:12:35). AAAAAABBBXXXXXT verisi RLE ile nasıl kodlanır ve sıkıştırma oranı nedir?

Çözüm (derste verildi): 6A 3B 5X 1T; 15 byte'tan 8 byte'a iner. Sıkıştırma oranı $15/8 = 1{,}875$ (ara adım eklendi; python ile doğrulandı). Aynı şekilde BBBBBBBBBAAAAAAAAAAAAAAAAANMMMMMMMMM → B09A16N01M10.

Soru 8 (ekrandan, 01:17:30). 13 sayısını slayttaki Punctured Elias adımlarıyla kodlayın.

Çözüm (derste verildi): $13 = 1101$; ters çevrilince 1011; 0001 bayrakları önüne eklenir ve en sağdaki bit atılır: $0001|101$. Bu dizi 13'ün Elias gamma koduyla aynıdır (python ile doğrulandı).

Soru 9 (ekrandan, 01:20:40). Goldbach G2 kodunda 1 ve 2 nasıl kodlanır? Tek bir n sayısının kodu nasıl kurulur?

Çözüm (derste verildi): 1 ve 2, 110 ve 111 olarak kodlanır; başka hiçbir kod 11 ile başlamaz. n tekse kod tek bir 1 ile başlar ve ardından $n - 1$ çift sayısının G2 kodu gelir.

Soru 10 (ekrandan, 01:31:30). 16 bitlik dinamik sabit nokta gösteriminde tamsayı kısmı 5 bitse kesirli kısım kaç bittir?

Çözüm (derste verildi): $frac_bits = total_bits - int_bits - 1 = 16 - 5 - 1 = 10$ bit (ara adım eklendi).

Soru 11 (ekrandan, 01:50:00). "the more I know about you the more I know about me" mesajı D-v2v ile ayrıştırılırken "you" kelimesine gelindiğinde bilinen diziler listesi nedir? Sonra "the", "more", "I" okununca ne olur?

Çözüm (derste verildi): S_0 "the", S_1 "more", S_2 "the more", S_3 "I", S_4 "more I", S_5 "know", S_6 "I know", S_7 "about", S_8 "know about", S_9 "you", S_{10} "about you". "the" ve "more" ile RS = "the more" olur; "I" gelince "the more I" listede olmadığı için S_2 gönderilir ve $S_{11} = S_9 \| S_2$ "you the more" eklenir.

Soru 12 (ekrandan, 01:55:30). Bölüm 7.5'teki durumda S_2 gönderildiğinde pos, top ve voc nasıl güncellenir?

Çözüm (derste verildi): freq[2] 1 olur; voc[2] ve voc[6] yer değiştirir (voc[2] = 6, voc[6] = 9); $pos[6] = 2$, $pos[9] = 6$; top[0] = 7.

Soru 13 (derste sözlü, 02:00:06). Sıkıştırma oranı tablosunda %35 değeri ne anlama gelir?

Çözüm (derste verildi): 100 birimlik veri 35 birime düşürülmüştür; yani %65'lik bir kazanç vardır. Değer başarım değil orandır, küçük olan daha iyidir.

Soru 14 (derste sözlü, 02:07:19). D-v2v sözlük tabanlı bir sıkıştırmaya dönüştürülebilir mi?

Çözüm (derste verildi): mümkündür, çünkü yöntem zaten kelime bazlı çalışır.

Dersin Toparlaması

Bu derste altı seminer sunumu yapıldı ve dönem boyunca görülen konuların uygulamadaki karşılıkları ortaya çıktı. Haar dalgacık dönüşümü, görüntüyü yaklaşık ve detay alt bantlarına ayırıp küçük detay katsayılarını sıfırlayarak sıkıştırır; DCT'den farkı bütün görüntüye uygulanması ve bloklanma bozulması üretmemesidir. Sinir ağı tabanlı kayıpsız sıkıştırıcılarda olasılığı bir LSTM ya da Transformer tahmin eder ve sembol aritmetik kodlayıcıyla kodlanır; enwik8'de gzip ve xz'den çok daha iyi oran verir, ama çok yavaştır. FastText.zip ise metni değil sınıflandırma modelini sıkıştırır: çarpım niceleme, dağarcık budama ve hashing ile modeli yüzlerce kat küçültürken doğruluk az düşer.

Tamsayı kodlarıyla ilgili sunumda RLE, aritmetik kodlama, Punctured Elias ve Goldbach kodları karşılaştırıldı; Goldbach kodları asal sayıların sıra numaralarını gamma koduyla yazar ve deneylerde en kısa çıktıyı vermiştir. CNN sıkıştırma sunumunda niceleme ile bit sayısı, budama ile işlem sayısı azaltıldı ve FPGA kaynaklarının nasıl değiştiği gösterildi. Son sunumdaki D-v2v, kelime tabanlı bir sembol listesiyle sık kelime dizilerine kısa kod verir; pos ve top dizileriyle sıralamayı ucuza korur ve İngilizce metinlerde p7zip'e yakın oranlara ulaşır. Hoca sunumlardan sonra makaleyi okuyup aktarmanın yetmediğini, makalenin anlaşılıp birkaç cümleyle özetlenebilmesi gerektiğini ve makale seçiminin önemli olduğunu vurguladı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.